

Process MeNtOR 3.0

Uni-SEP

Legends of Sword and Wand Design Document

Version:	1.0
Print Date:	01/31/2026
Release Date:	02/27/2026
Release State:	Draft
Approval State:	Pending
Approved by:	N/A
Prepared by:	Ethan Persaud, Mohammed Patel, Selimalp Bayrak, William Bui
Reviewed by:	N/A
Path Name:	/Deliverable 1/Legends of Sword and Wand Report
File Name:	LegendsOfSwordAndWand_Design Document.pdf
Document No:	LSW-DD-01

Document Change Control

Version	Date	Authors	Summary of Changes
1.0	02/03/2026	Entire Team	Initial Design Document

Document Sign-Off

Name (Position)	Signature	Date
Mohammed Patel (Developer)	Mohammed Patel	02/27/2026
Selimalp Bayrak (Developer)	Selimalp Bayrak	02/27/2026
Ethan Persaud (Developer)	Ethan Persaud	02/27/2026
William Bui (Developer)	William Bui	02/27/2026

Contents

1	INTRODUCTION
1.1	Purpose
1.2	Overview
1.3	Resources - References
2	SEQUENCE DIAGRAMS
3	MAJOR DESIGN DECISIONS
3.1	Layered Architecture
3.2	Domain-Driven Modularization
3.3	Use of Interfaces
3.4	Strategy-Based Battle Execution
3.5	Database Abstraction with Spring Data JPA
3.6	Prevention of God Classes
4	ARCHITECTURE
4.1	Component Diagram
4.2	Interface Specification
4.3	Architectural Patterns Used
5	DETAILED CLASS DIAGRAMS
5.1	UML Class Diagrams
6	USE OF DESIGN PATTERNS
6.1	Strategy Pattern
6.2	Factory Pattern
6.3	Singleton Pattern
6.4	Template Method Pattern
6.5	Observer Pattern
6.6	Decorator Pattern
7	ACTIVITIES PLAN
7.1	Gantt Chart
7.1	Project Backlog and Sprint Backlog
7.2	Group Meeting Logs
8	TEST DRIVEN DEVELOPMENT

1. Introduction

1.1. Purpose

This document details the architectural and design specifications of the *Legends of Sword and Wand* system.

It describes the structural decomposition, component interactions, class design and design patterns employed in the implementation of the system.

The purpose of this document is to serve as a technical reference for developers, testers and stakeholders involved in the development, validation and maintenance of the system. It ensures architectural consistency, traceability to use cases and alignment with project requirements.

GitHub Repository:

<https://github.com/MohammedPatel-yorku/LegendsOfSwordAndWand>

1.2. Overview

Legends of Sword and Wand is a modular tactical role-playing game system implemented using Spring Boot and MySQL. The system supports:

- User profile creation and authentication
- Player-versus-Environment campaign progression
- Turn-based battle mechanics
- Persistent storage of party, hero and campaign state

The system is designed using a layered architecture that separates presentation logic, business logic and data persistence. This design promotes scalability, maintainability and adherence to high cohesion and low coupling principles.

This design document is organized as follows:

- **Cover:** Main Page and document revision history
- **Section 1:** Introduction and project context
- **Section 2:** Major architectural and modularization decisions
- **Section 3:** Sequence diagrams for all primary use cases
- **Section 4:** System architecture and module decomposition
- **Section 5:** UML class diagrams for implemented components
- **Section 6:** Design patterns and justification
- **Section 7:** Project planning, backlog and Gantt chart
- **Section 8:** Test-Driven Development strategy and test cases

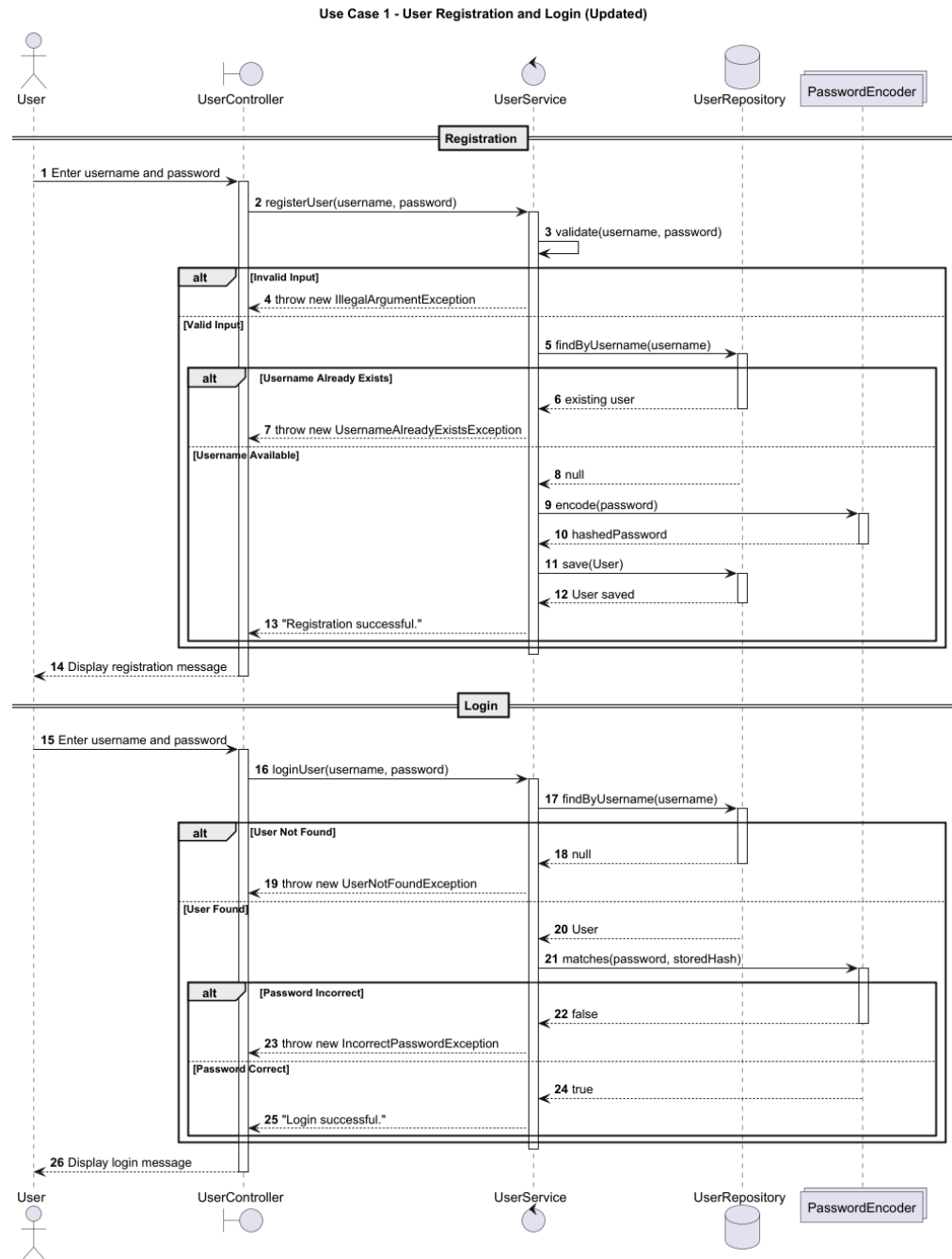
1.3. Resources - References

The following resources were used to aid the development of the system:

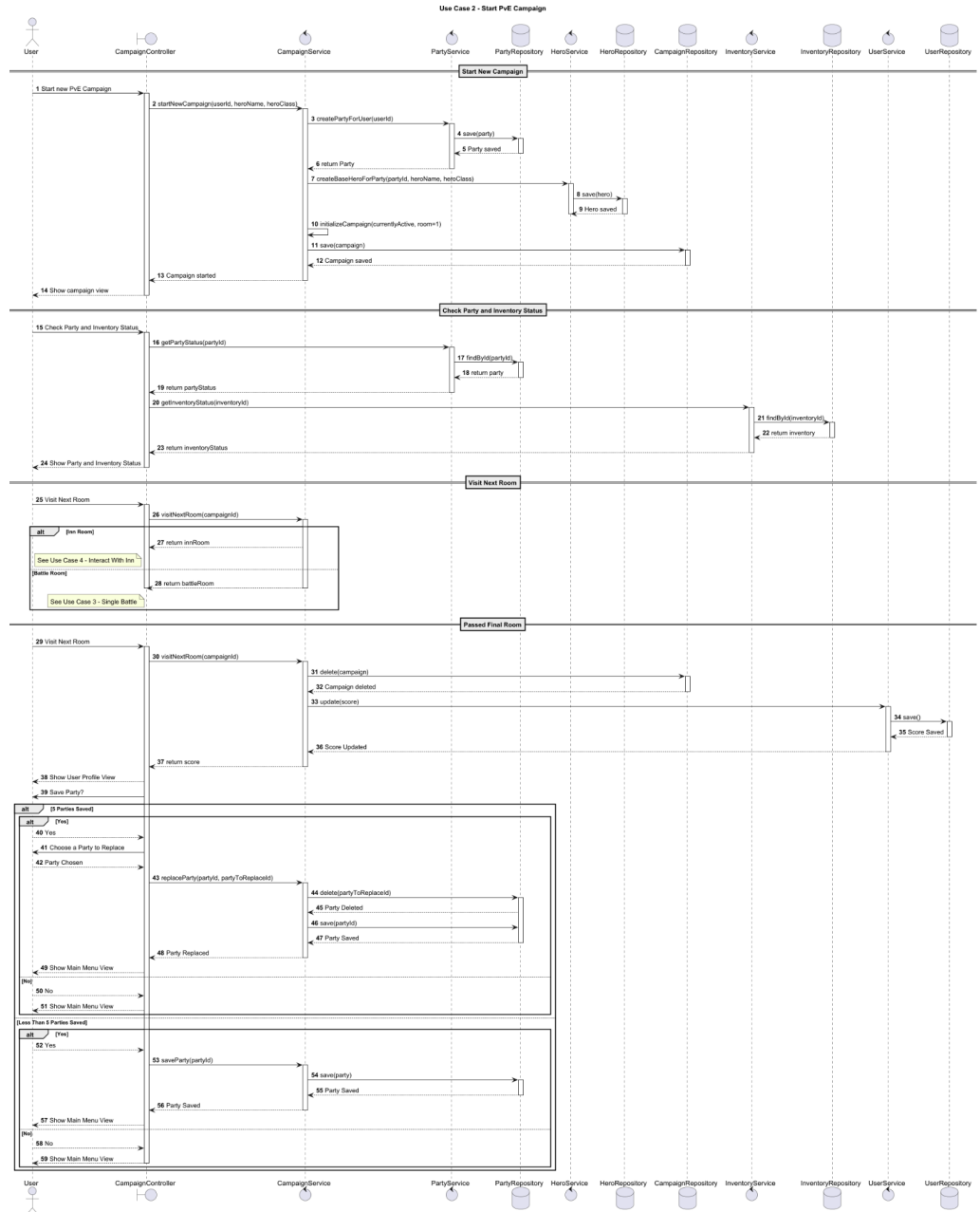
- “Project Description - Legends of Sword and Wand”
- “Deliverable 1: Instructions”
- “Project Requirements: Use Cases”
- Spring Boot Documentation
- Spring Data JPA Documentation
- UML 2.x Specification

2. Sequence Diagrams

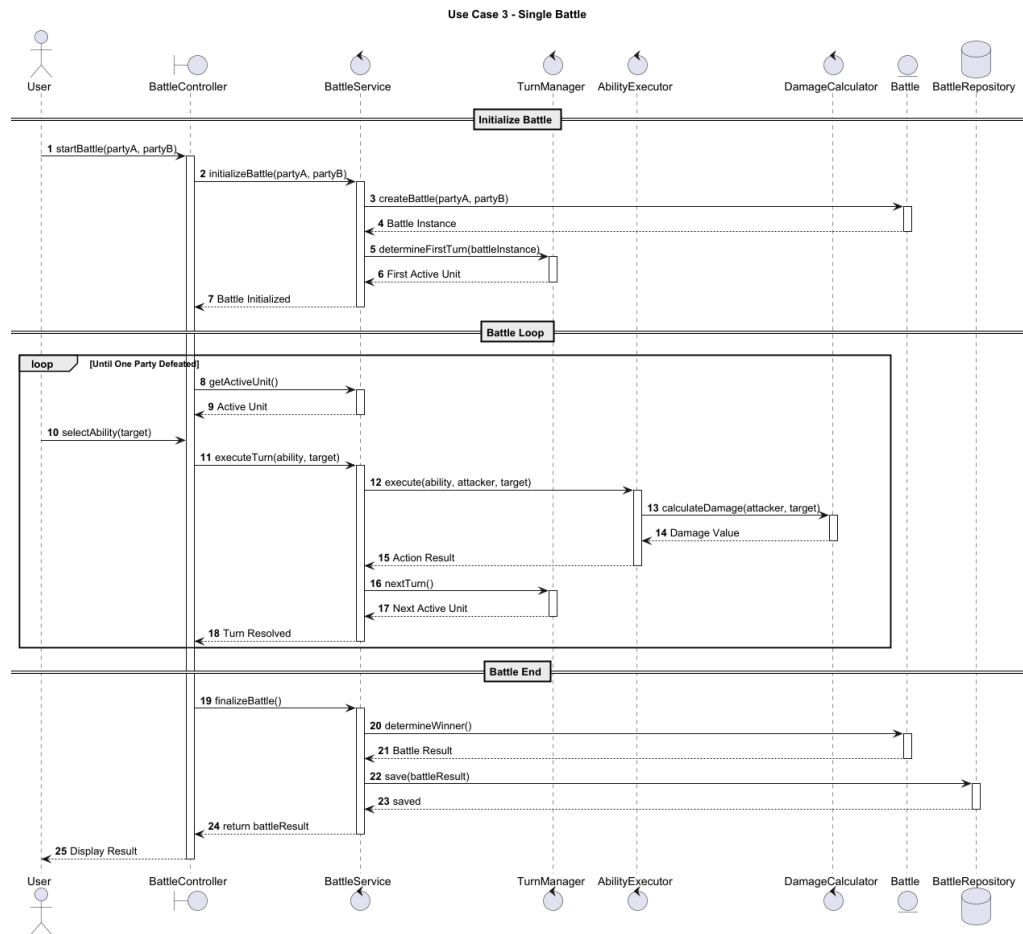
- Use Case 1: User Registration and Login



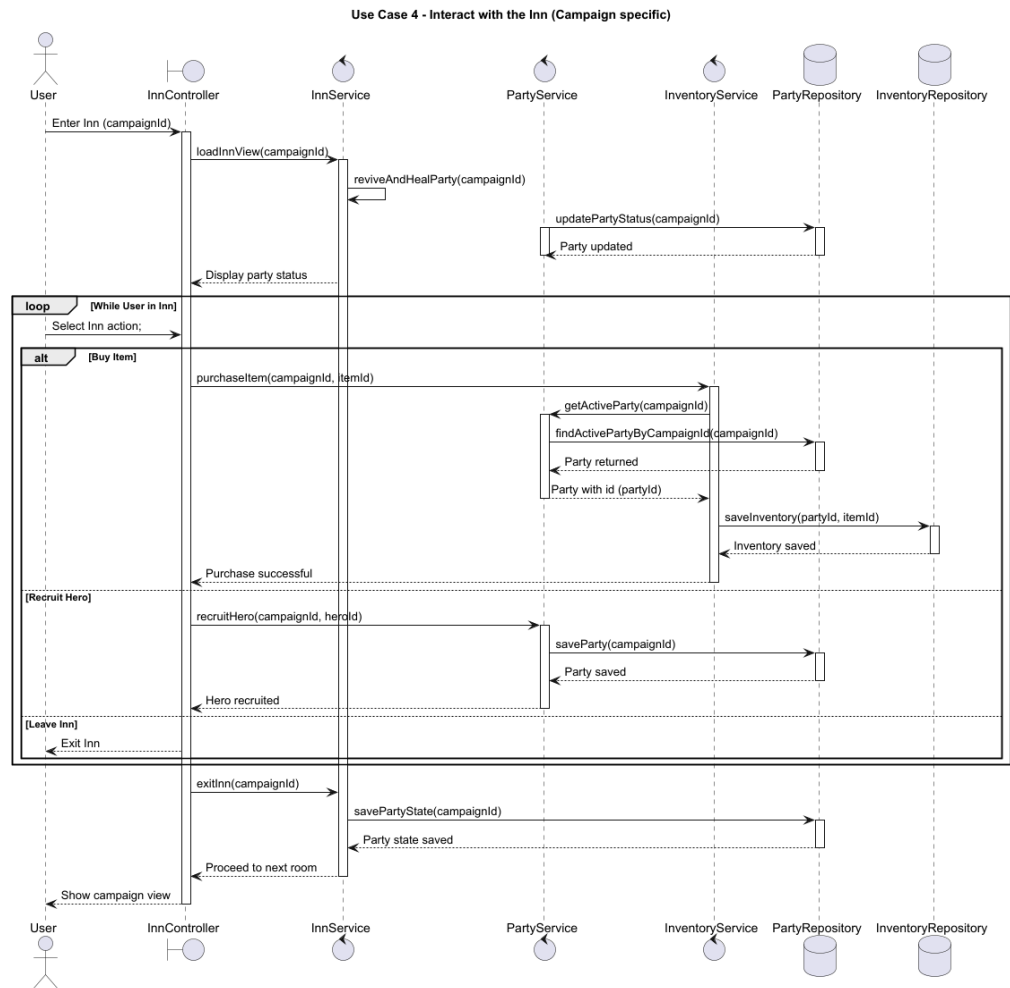
● Use Case 2: Start PvE Campaign



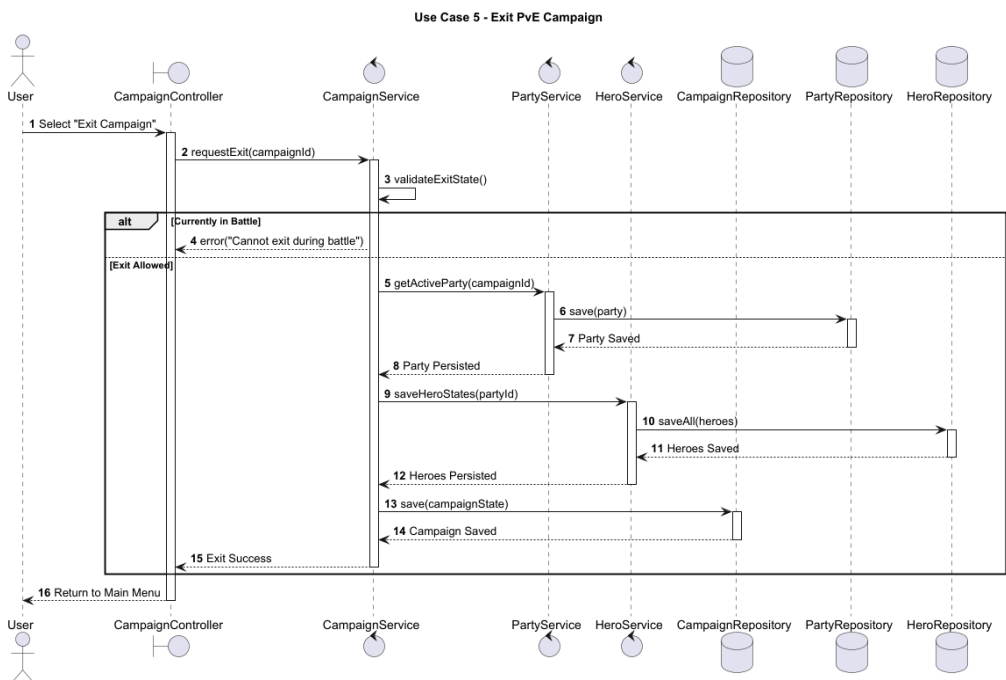
- Use Case 3: Single Battle



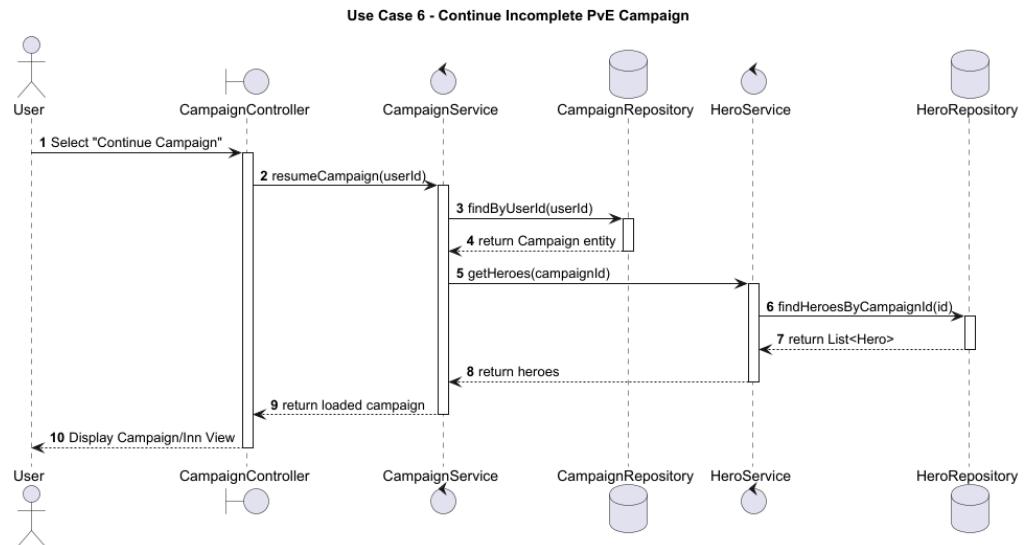
- Use Case 4: Interact with the Inn



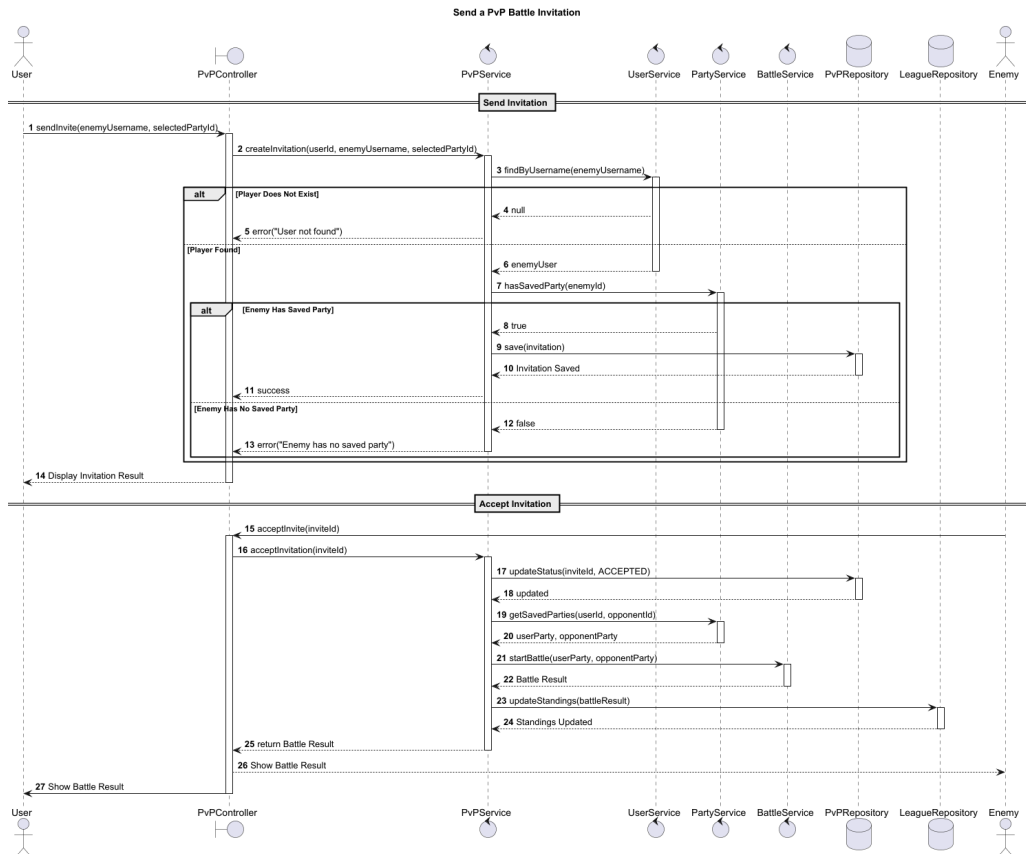
- Use Case 5: Exit PvE Campaign



- Use Case 6: Resume Campaign



- Use Case 7: Invite to PvP Battle



3. Major Design Decisions

The design decisions made for the Legends of Sword and Wand system were based on modularity, maintainability, extensivity and a clear separation of concerns.

The system follows an architecture that separates presentation, business logic and persistence concerns, enabling the development and testing of each, individual layer and component.

3.1. Layered Architecture

The system uses a layered architecture consisting of:

- Presentation Layer (Controllers)
- Application Layer (Services)
- Persistence Layer (Repositories)
- Domain Layer (Entities)

This design was selected to ensure high cohesion and low coupling.

Controllers handle user interaction only.

Services encapsulate business logic.

Repositories abstract database access.

Entities represent domain state.

This separation of concerns ensures that modifications to one layer do not affect the rest of the system.

3.2. Domain-Driven Modularization

The system is decomposed into modules aligned with core game domains:

- User Module
- Hero Module
- Party Module
- Battle Module
- Campaign Module
- PvP Module
- Persistence Module

Each module encapsulates a single responsibility. For example:

- The Battle Module handles only combat logic.
- The Campaign Module controls PvE progression but does not implement Battle mechanics.

This reduces cross-module dependencies and enforces modular growth.

3.3. Use of Interfaces

All services are defined through interfaces (e.g., IBattleService, ICampaignService).

Concrete implementations depend on abstractions, not on other concrete classes.

This achieves:

- Loose coupling
- Testability
- Easier future refactoring
- Clear dependency injection using Spring Boot

3.4. Strategy-Based Battle Execution

Battle actions are implemented using the Strategy pattern.

This decision prevents conditional explosion inside a single Battle class.

Each action is encapsulated in its own class implementing ActionStrategy.

This improves:

- Extensibility
- Readability
- Adherence to Open-Closed Principle

3.5. Database Abstraction with Spring Data JPA

The system uses Spring Data JPA repositories for persistence.

Repositories:

- Extend JpaRepository
- Contain no business logic
- Are injected into services

This design ensures:

- Database independence
- Simplified CRUD operations
- Proper separation of concerns

3.6. Prevention of God Classes

Special attention was paid to prevent large monolithic classes.

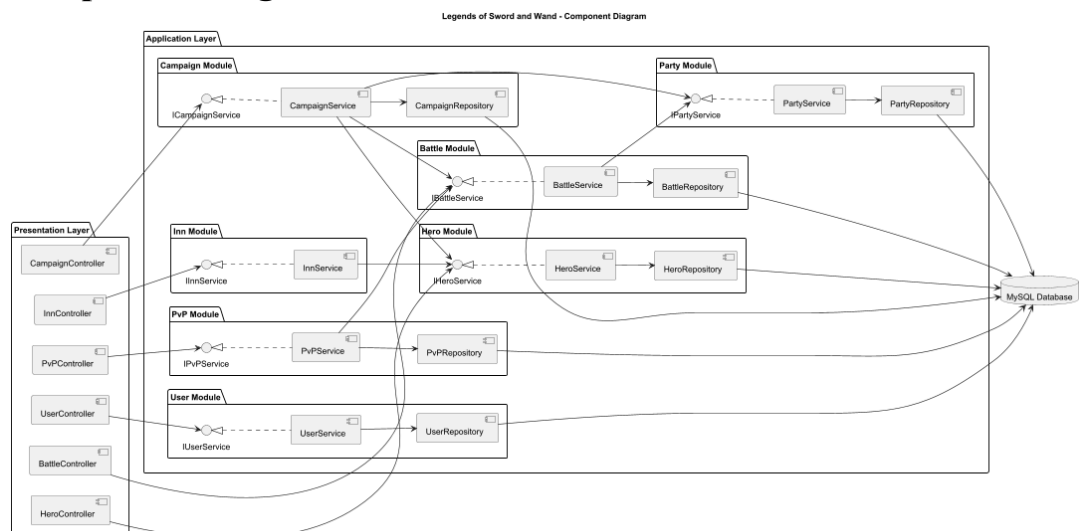
For example:

- Battle orchestration is separate from action execution.
- Campaign flow is separate from Battle resolution.
- Hero leveling is isolated from persistence logic.

This improves maintainability and readability.

4. Architecture

4.1. Component Diagram



Component Name	Description
User Module	Handles User registration, authentication and profile persistence.

Hero Module	Manages Hero creation, leveling, class bonuses and hybrid logic.
Party Module	Manages Party composition, Hero membership and gold tracking.
Battle Module	Implements the Battle engine, turn logic and action execution.
PvP Module	Controls PvP flow, including initiation and Battle interactions.
Campaign Module	Controls PvE flow, room progression and score calculation.
Inn Module	Manages Inn interactions, such as Hero regeneration and Item purchases.

4.2. Interface Specification

<i>Interface Name</i>	<i>Signature of Operations Exposed in Interface</i>	<i>Description of the Operation</i>
IUserService	void register(String username, String password)	Creates a new User profile and persists it to the database.
	User login(String username, String password)	Authenticates a User and returns the corresponding profile object.
IHeroService	Hero createHero(String name, HeroClass classType)	Instantiates a Hero with base stats and selected class.
	void levelUp(Hero hero)	Applies level growth and class bonuses.
IPartyService	Party createParty(User user)	Creates a new Party for a User.
	addHeroToParty(Party party, Hero hero)	Adds a Hero to a Party if capacity allows.
IBattleService	Battle initiateBattle(Party p1, Party p2)	Creates Battle instance and determines turn order.
	performAction(Hero actor, ActionStrategy strategy, Hero target)	Executes selected strategy.
IPvPService	sendInvite(userId, enemyId, selectedParty)	Sends a PvP invitation to the opposing player.
	acceptInvite(selectedParty)	Accepts a PvP invitation sent to User

	initiateBattle(partyA, partyB)	Initiates a PvP Battle with selected Parties.
ICampaignService	Campaign startCampaign(User user, Hero hero)	Initializes PvE Campaign.
	void visitNextRoom(Campaign campaign)	Determines whether next room is Battle or Inn.
	void saveProgress(Campaign campaign)	Persists Campaign state to database.

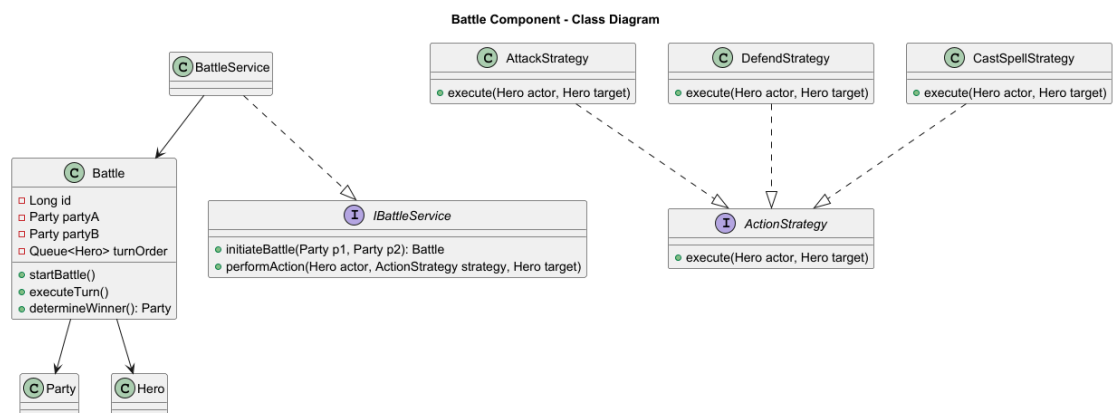
4.3. Architectural Patterns Used

The system follows a layered architectural pattern consisting of Presentation, Application and Persistence layers. This pattern was selected to ensure clear separation of concerns, maintainability and scalability. Controllers handle incoming requests, service interfaces define the application's business logic boundaries, service implementations coordinate domain operations and repositories manage database persistence using Spring Data JPA with MySQL. This structure promotes modularity and ensures that changes in one layer do not directly affect others.

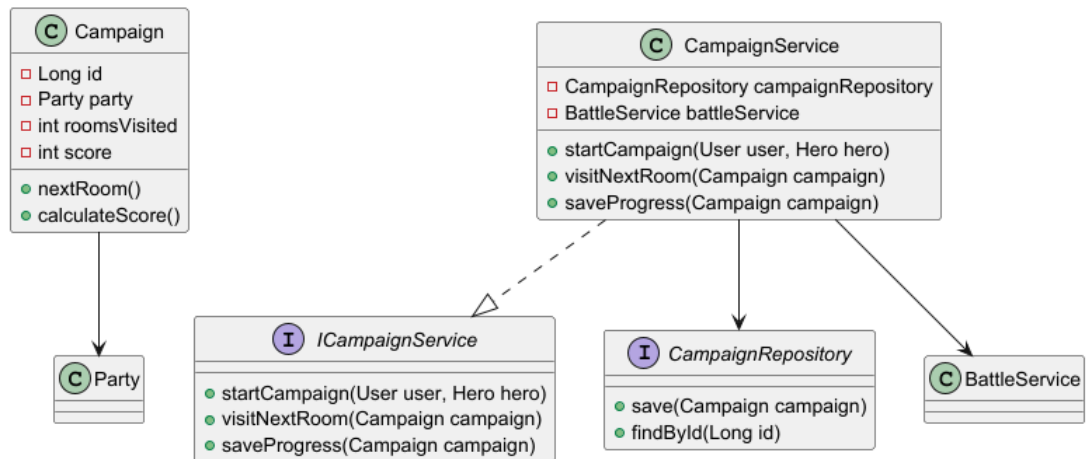
Additionally, the system adopts a modular service-oriented structure where each functional area (User, Hero, etc.) is encapsulated within its own module and exposes a single service interface. This approach supports loose coupling between modules and enables reuse of the Battle module by both Campaign and PvP components. By enforcing dependency on interfaces rather than concrete implementations, the design adheres to the Dependency Inversion Principle, improving testability and architectural clarity.

5. Detailed Class Diagrams

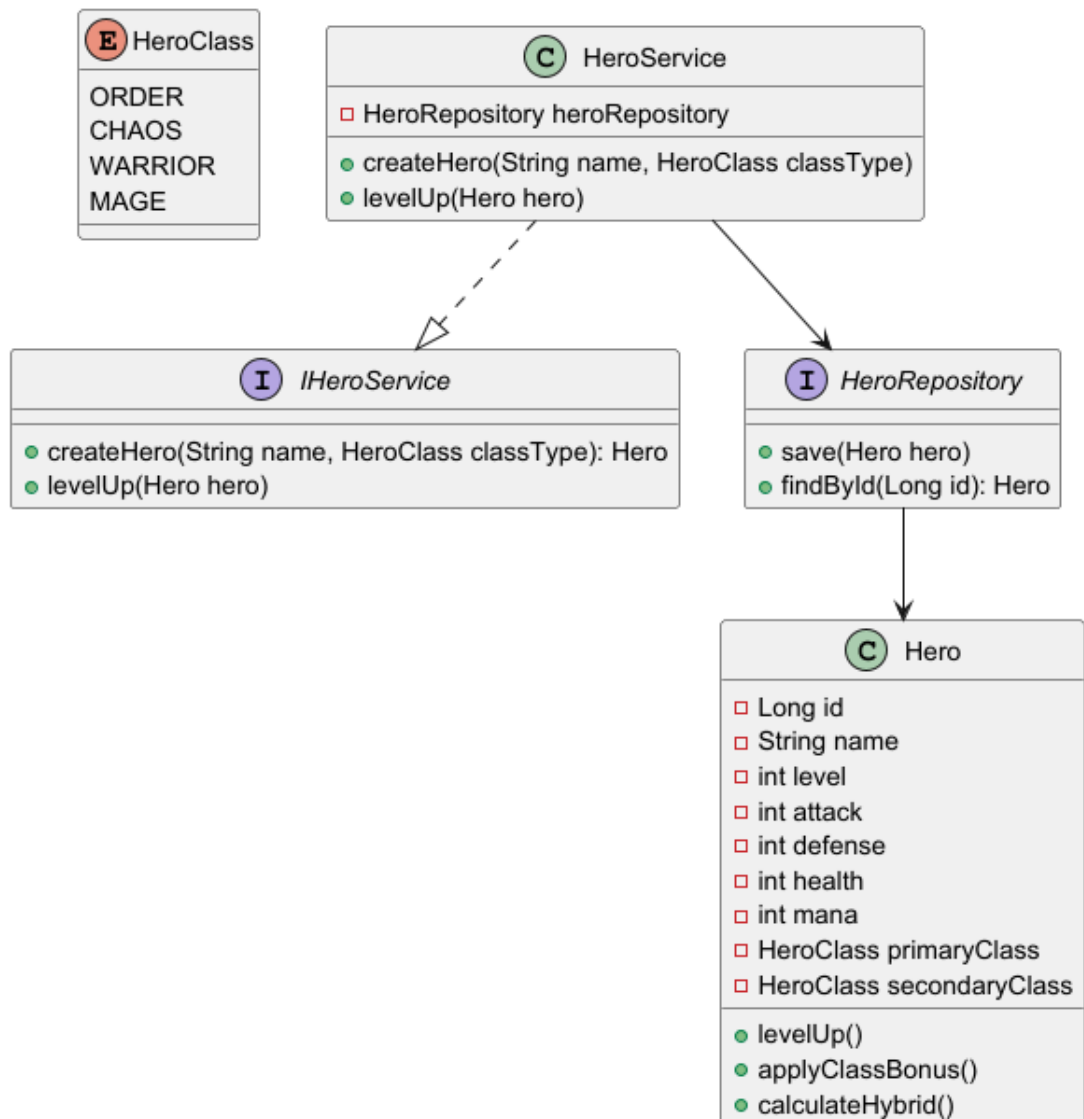
5.1. UML Class Diagrams



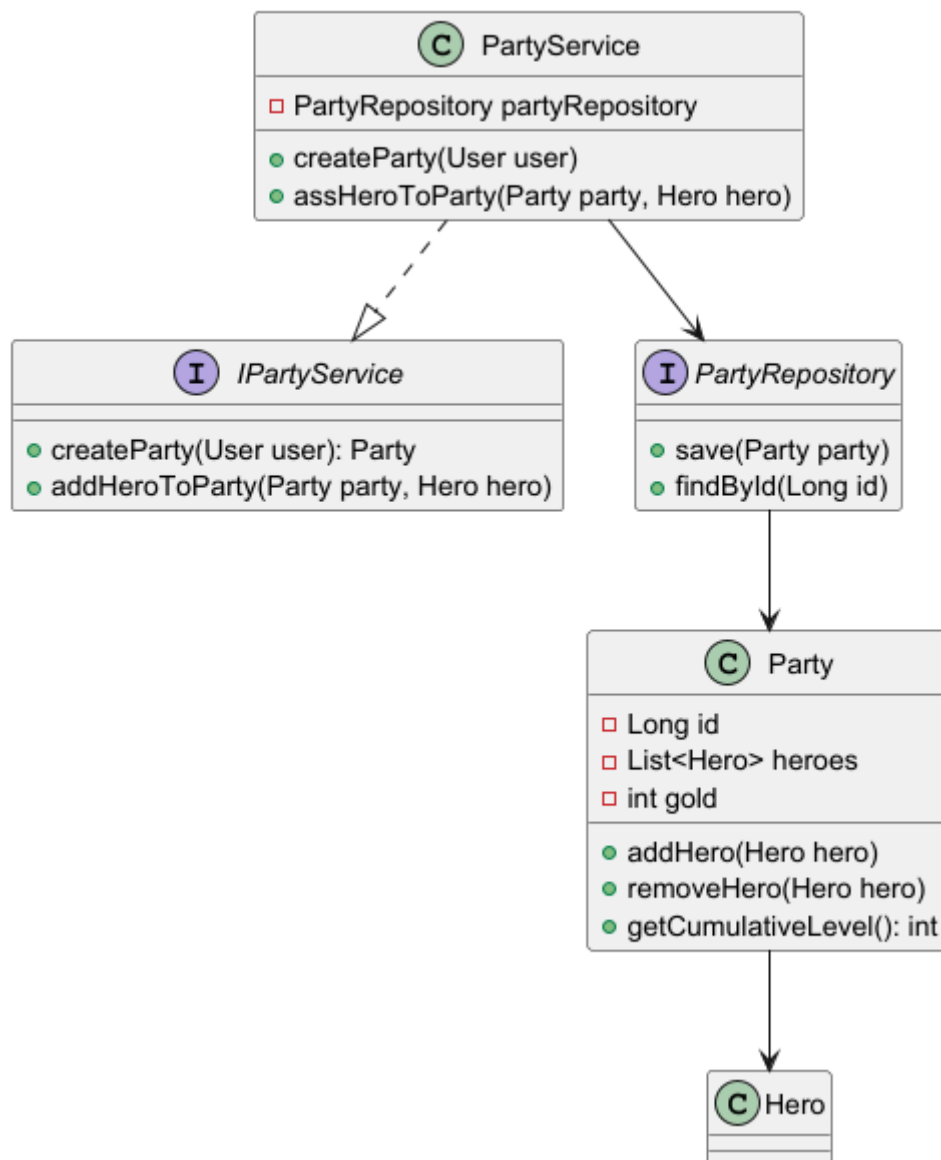
Campaign Component - Class Diagram



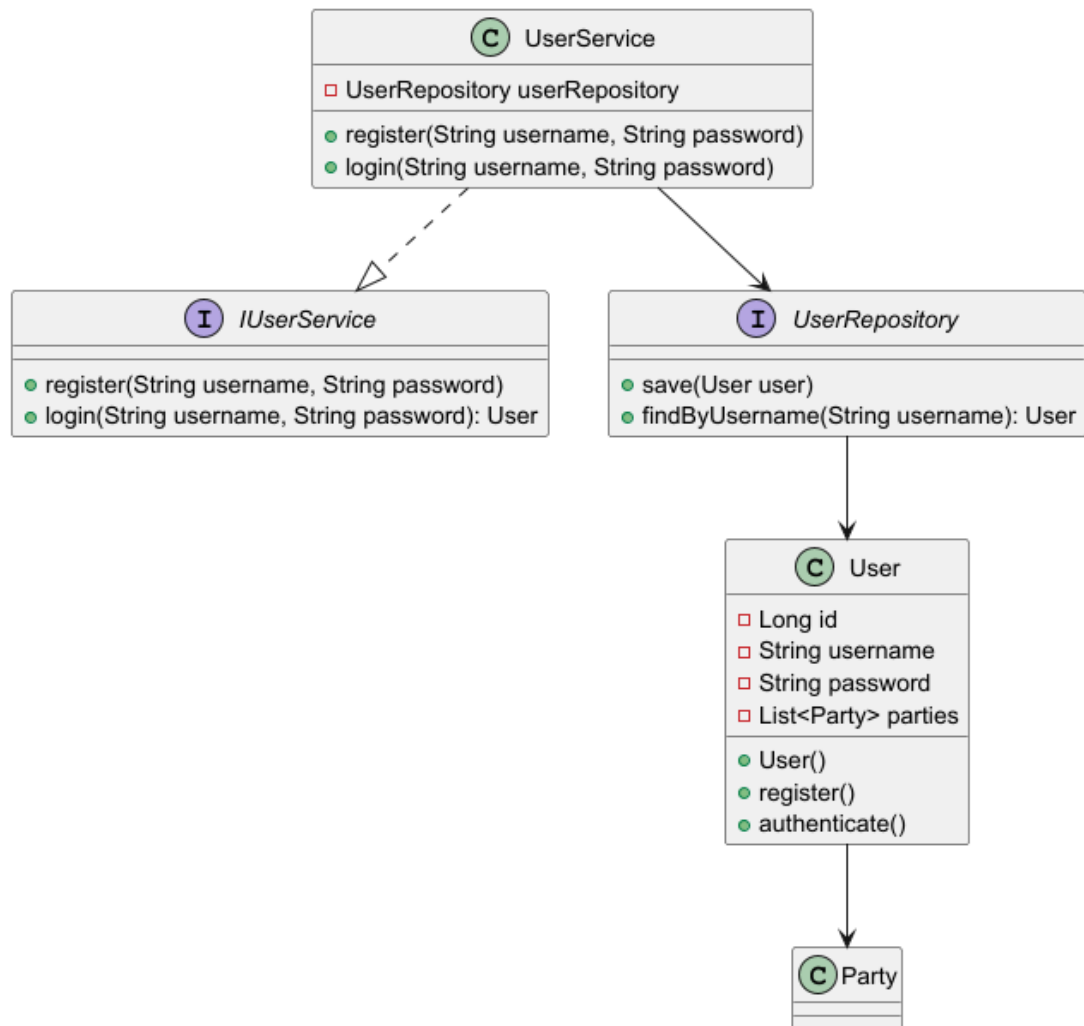
Hero Component - Class Diagram



Party Component - Class Diagram



User Component - Class Diagram



6. Use of Design Patterns

The following design patterns are implemented or planned to be fully implemented by Deliverable 3.

6.1. Strategy Pattern

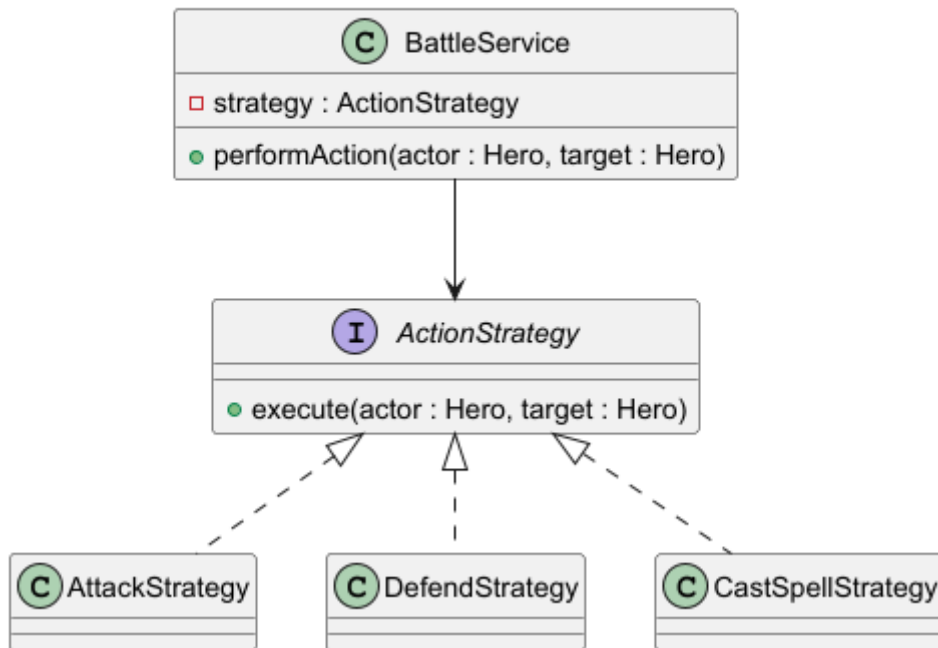
Location: Battle Module

Classes: ActionStrategy, AttackStrategy, DefendStrategy, CastSpellStrategy

Purpose: Encapsulates different Battle actions and allows them to vary independently.

Why Chosen: Prevents large conditional blocks in the Battle engine and adheres to the Open-Closed Principle.

Battle Actions - Strategy Pattern



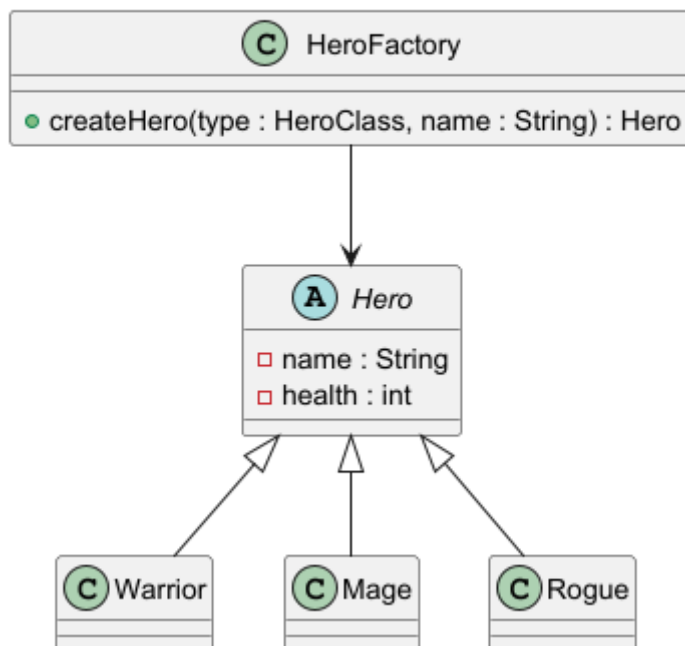
6.2. Factory Pattern

Location: Battle Module

Purpose: Centralizes Hero creation and ensures correct base stats and class bonuses are applied.

Why Chosen: Avoids duplicating Hero initialization logic across Controllers and Services.

Hero Creation - Factory Pattern



6.3. Template Method Pattern

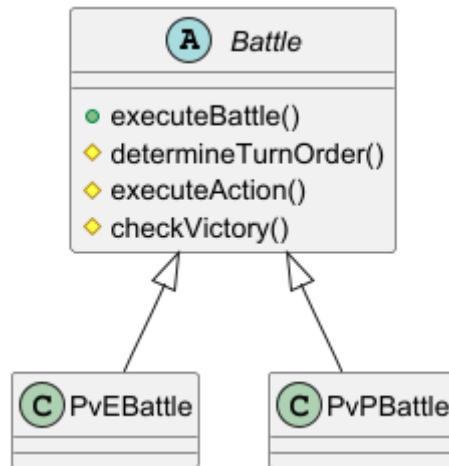
Location: Battle Engine

The Battle flow will follow a fixed structure:

1. Determine turn order
2. Execute action
3. Apply effects
4. Check victory

Subclasses or strategies override specific steps.

Purpose: Ensures Battle flow remains consistent while allowing variation in execution details.



6.4. Observer Pattern

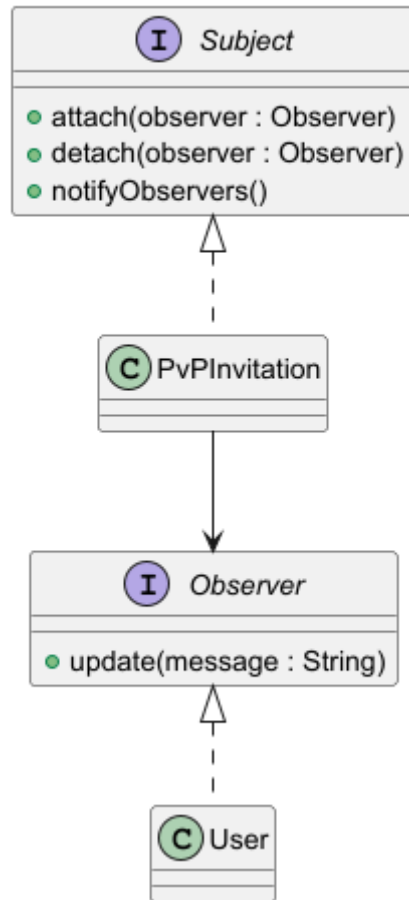
Location: PvP Module

Used for notifying players when:

- Invitation is sent
- Invitation is accepted
- Battle result is determined

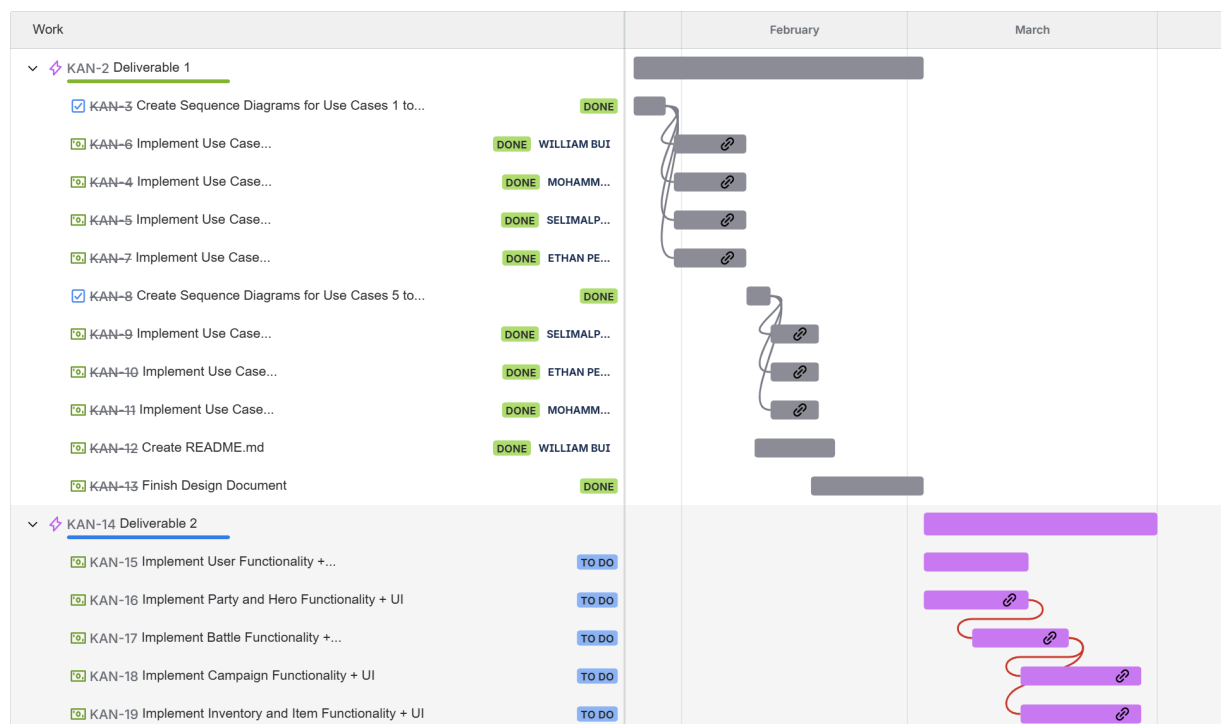
Purpose: Decouples PvP match events from User notification logic.

PvP Battle - Observer Pattern



7. Activities Plan

7.1. Gantt Chart



7.2. Project Backlog and Sprint Backlog

Project BacklogSprint Backlog (Jan 29, 2026 - Feb 20, 2026)

ID	Task Description	Deliverable	Due Date	Status	Assigned To
KAN-3	Create Sequence Diagrams for Use Cases 1 to 4	Deliverable 1	Jan 29, 2026	Done	Team
KAN-4	Implement Use Case 2	Deliverable 1	Feb 8, 2026	Done	Mohammed
KAN-5	Implement Use Case 3	Deliverable 1	Feb 8, 2026	Done	Selim
KAN-6	Implement Use Case 1	Deliverable 1	Feb 8, 2026	Done	Will
KAN-7	Implement Use Case 4	Deliverable 1	Feb 8, 2026	Done	Ethan
KAN-8	Create Sequence Diagrams for Use Cases 5-7	Deliverable 1	Feb 11, 2026	Done	Team
KAN-9	Implement Use Case 5	Deliverable 1	Feb 17, 2026	Done	Selim
KAN-10	Implement Use Case 6	Deliverable 1	Feb 17, 2026	Done	Ethan
KAN-11	Implement Use Case 7	Deliverable 1	Feb 17, 2026	Done	Mohammed
KAN-12	Create README.md	Deliverable 1	Feb 19, 2026	Done	Will
KAN-13	Finish Design Document	Deliverable 1	Mar 2, 2026	Done	Team
KAN-15	Implement User Functionality + UI	Deliverable 2	Mar 15, 2026	To Do	Unassigned
KAN-16	Implement Party and Hero Functionality + UI	Deliverable 2	Mar 15, 2026	To Do	Unassigned
KAN-17	Implement Battle Functionality + UI	Deliverable 2	Mar 20, 2026	To Do	Unassigned
KAN-18	Implement Campaign Functionality + UI	Deliverable 2	Mar 29, 2026	To Do	Unassigned
KAN-19	Implement Inventory and Item Functionality + UI	Deliverable 2	Mar 29, 2026	To Do	Unassigned

Task ID	Task Description	Due Date	Status	Assigned To
KAN-3	Create Sequence	Jan 29, 2026	Done	Team

	Diagrams for Use Cases 1–4			
KAN-4	Implement Use Case 2	Feb 8, 2026	Done	Mohammed
KAN-5	Implement Use Case 3	Feb 8, 2026	Done	Selim
KAN-6	Implement Use Case 1	Feb 8, 2026	Done	Will
KAN-7	Implement Use Case 4	Feb 8, 2026	Done	Ethan
KAN-8	Create Sequence Diagrams for Use Cases 5–7	Feb 11, 2026	Done	Team
KAN-9	Implement Use Case 5	Feb 17, 2026	Done	Selim
KAN-10	Implement Use Case 6	Feb 17, 2026	Done	Ethan
KAN-11	Implement Use Case 7	Feb 17, 2026	Done	Mohammed
KAN-12	Create README.md	Feb 19, 2026	Done	Will
KAN-13	Finish Design Document	Mar 2, 2026	In Progress	Team

7.3. Group Meeting Logs

Present Group Members	Meeting Date	Issues Discussed / Resolved
Ethan, Mohammed, Will, Selim	January 27, 2026	- Discuss resources needed for project; what to install, setup - Discuss project deliverable 1 requirements - Began configuring mySQL - Set up next meeting
Ethan, Mohammed, Will, Selim	January 28, 2026	- Set up mySQL database connection to intellij
Ethan, Mohammed, Will, Selim	February 2, 2026	- Discussed work delegation - Discussed including Javadoc
Ethan, Mohammed, Will, Selim	February 10, 2026	- Discuss where everyone is at in their implementation - Assigned more work to people (more use cases)
Ethan, Mohammed, Will, Selim	February 11, 2026	- Discussed how we will make our component diagram - We each work on what we have implemented then put it all together

Ethan, Mohammed, Will	February 18, 2026	- Discussed our priorities for deliverable 1 - Decided to focus more on having a polished report more importantly than having actual code implementation - Aim to change our sequence diagrams to more accurately represent our design - Ensure when implementing to the main classes of each use case, implement detailed exception handling reports - Discuss how we are going to make the component diagram - We will probably all get together and work on it together - Discussed design patterns used to add to our report - Created a rough component diagram to give a better idea of what the fleshed out version would look like - Worked on adding the design patterns we used in our code, to the report
Ethan, Mohammed, Will, Selim	February 26, 2026	- Discussed completed component diagram + class diagram - Discussed report progress - Discussed work delegation for more detailed code implementation - Discussed adding runnable tests to the use cases for deliverable 1 - Discussed how we are going to implement our Gantt chart and project/sprint backlogs

8. Test Driven Development

USE CASE 1

Test ID	Case 1-01
Category	successful flow
Requirements Coverage	UC1 – successful user registration
Initial Condition	the system is running the user is not logged in the username does not already exist in the system
Procedure	the user selects “Register” the user enters a valid username

	the user enters a valid password the user submits the registration form
Expected Outcome	the system validates the input the password is hashed a new user record is created the system confirms successful registration
Notes	Username must be unique (no repeats). Password must meet minimum security requirements. (if we add one probably not though)

Test ID	Case 1-02
Category	duplicate username
Requirements Coverage	UC1 – username must be unique
Initial Condition	the system is running a user with username "player1" already exists
Procedure	the user selects “Register” the user enters username "player1" the user enters a valid password the user submits the form
Expected Outcome	the system detects duplicate username registration is rejected an error message is displayed no new user is created
Notes	To prevent multiple accounts with the same name

Test ID	Case 1-03
Category	successful authentication
Requirements Coverage	UC1 – successful login
Initial Condition	user account exists correct username and password are known
Procedure	user selects “login” user enters correct username user enters correct password user submits form
Expected Outcome	system checks credentials user is correct user is redirected to main system interface
Notes	

Test ID	Case 1-04
Category	incorrect password
Requirements Coverage	UC1 – authentication failure handling
Initial Condition	user account exists
Procedure	user selects “Login” user enters correct username user enters incorrect password user submits form
Expected Outcome	Check fails login unsuccessful message displayed user remains logged out
Notes	

USE CASE 2

Test ID	Case 2-01
Category	campaign creation
Requirements Coverage	UC2 – Successful Campaign Creation
Initial Condition	system is running user is registered and logged in user does not have an active campaign
Procedure	the user selects "start new campaign." the system prompts for the hero's name and the hero's class the user enters a valid hero name the user selects a valid hero class the user confirms creation
Expected Outcome	a new campaign is created the campaign is marked as active the campaign starts in room 1 a new party is created a starting hero is created with the selected name and class
Notes	hero name must meet validation rules

Test ID	Case 2-02
----------------	-----------

Category	campaign initial state validation
Requirements Coverage	UC2 – campaign initialization rules
Initial Condition	user is logged in
Procedure	the user starts a new campaign the system creates the campaign
Expected Outcome	Campaign.active = true Campaign.currentRoom = 1
Notes	

Test ID	Case 2-03
Category	user validation
Requirements Coverage	UC2 – user must exist
Initial Condition	system running invalid or non-existent user ID
Procedure	A campaign start is attempted with an invalid user ID
Expected Outcome	the system rejects the request an error message or exception is generated no campaign is created
Notes	

Test ID	Case 2-04
Category	input validation – hero name
Requirements Coverage	UC2 – hero name validation
Initial Condition	User logged in
Procedure	user selects Start Campaign user leaves hero name empty user confirms
Expected Outcome	campaign creation is rejected validation error is displayed no campaign is created
Notes	

USE CASE 3

Test ID	Case 3-01
Category	Battle initialization
Requirements Coverage	UC3 – successful battle initialization
Initial Condition	System running User logged in Campaign is active Player and enemy party are initialized Player and enemy party both have at least one alive hero
Procedure	User enters battle room
Expected Outcome	New battle instance created Turn order initialized Battle status set to active
Notes	Initialization has to happen before action executions Turn queue has to include all alive units from both parties

Test ID	Case 3-02
Category	Attack execution
Requirements Coverage	UC3 – execute attack action
Initial Condition	Battle is active Turn order initialized Active unit is alive Target unit is alive
Procedure	User selects <i>Attack</i> User selected an enemy hero
Expected Outcome	Damage calculated properly Target unit hp calculated properly Hp does not go below 0 Turn completes
Notes	

USE CASE 4

Test ID	Case 4-01
Category	Inn entry and party update
Requirements Coverage	UC4 – enter inn & restore party
Initial Condition	System is running User is logged in

	Campaign is active Party contains at least one damage or defeated hero
Procedure	User enters inn
Expected Outcome	All heroes are revived / healed Updated party saved to database Inn view shows updated statuses of party
Notes	

Test ID	Case 4-02
Category	Purchase item
Requirements Coverage	UC4 – purchase item
Initial Condition	User is in inn Party has enough gold Item is available to purchase
Procedure	User selects <i>buy item</i> User chooses certain item from item list
Expected Outcome	Item is added to party inventory Gold reduced from party based on item's price Purchase confirmation displayed
Notes	If the party doesn't have enough gold, there should be a rejection text

USE CASE 5

Test ID	Cse 5-01
Category	Campaign Exit – Successful Exit
Requirements Coverage	UC5 – user can exit active campaign
Initial Condition	system is running user is logged in user has an active PvE campaign
Procedure	the user selects “Exit Campaign” the system prompts for confirmation the user confirms exit
Expected Outcome	the campaign is saved campaign remains stored in database campaign.active is set to false (if required by design) user is returned to main menu/dashboard
Notes	campaign progress must not be lost.

Test ID	Cse 5-02
Category	Validation – No Active Campaign
Requirements Coverage	UC5 – exit only active campaign
Initial Condition	user is logged in user does not have an active campaign
Procedure	user tries to exit campaign
Expected Outcome	system prevents operation error message displayed no changes occur
Notes	

USE CASE 6

Test ID	Cse 6-01
Category	campaign resume – successful resume
Requirements Coverage	UC6 – resume existing campaign
Initial Condition	system running user logged in user has previously saved campaign
Procedure	user selects “Resume Campaign” system loads saved campaign
Expected Outcome	campaign is loaded campaign.active = true user is placed in last saved room party and hero states restored
Notes	

Test ID	Cse 6-02
Category	validation – no saved campaign
Requirements Coverage	UC6 – resume requires existing campaign
Initial Condition	user has no saved campaign
Procedure	user selects “Resume Campaign”
Expected Outcome	system displays error message no campaign is loaded
Notes	

USE CASE 7

Test ID	Case 7-01
Category	PVP invitation - sending invitation
Requirements Coverage	UC7 – send pvp invite
Initial Condition	System is running Inviting user is logged in Inviting user has at least one saved party Invited username exists in database Invited user has at least one saved party Invited user is not currently in battle
Procedure	User selects pvp mode User enters username of another player User selects send invitation
Expected Outcome	System validates that invited username exists System verifies both users have at least one saved party Invitation is created and sent to invited user Inviting user sees confirmation message Invited user receives notification of pvp invite
Notes	The username entered is case sensitive